

[Task-oriented WGs](#) > [Working Group IV – Common Analysis](#) >

[WG IV] Interactive Visualization of Astrophysical Data

The following content is originally created by Alex Bogert, John Holdener, and Nicholas Smith (Dept. of Physics and Computer Science, UCSC) in collaboration with Joel Primack and Matt Turk, for the GPU Technology Conference, Mar. 2014.

Contents

- [1 The yt Project](#)
- [2 PyCUDA](#)
- [3 Volume Rendering with CUDA](#)
- [4 IPython Notebook](#)
- [5 PyOpenGL](#)
- [6 Stereoscopic Rendering](#)

[The yt Project](#)

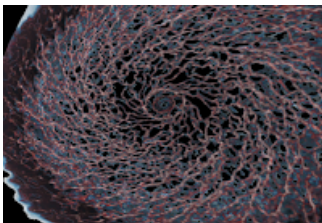
yt [1] is an analysis and visualization system for astrophysical volumetric data that is openly developed and freely available. At its core, yt provides a method of describing physical rather than computational objects inside an astrophysical simulation. yt provides methods for selecting regions, applying analysis to regions, visualizing (including volume rendering, projections, slices, phase plots) and exporting data to external analysis packages.

As an example, Adaptive Mesh Refinement (AMR) data consists of cells or grid patches that may be refined to higher-resolution and overlap with coarser objects. yt, when invoked on an AMR dataset, will transparently select data by masking the coarse cells, converting code units to physical units, process that data, optionally decomposing to multiple processors for parallel analysis, and then return the reduced data product or visualization to the user. yt allows astrophysicists to think about physical questions that drive their research, rather than necessary computational and technical steps to answer these questions.

yt has been designed to enable separation between data indexing and selection (such as patch-based AMR, N-body datasets, octrees, and irregular or unstructured meshes) and the processing of data to produce astrophysically-meaningful outputs. This enables researchers to utilize identical routines for generating simulated spectra and observations independent of the underlying simulation code, and provides direct cross-code comparisons and technology transfer. A fundamental component of yt is its investment in community: by providing different levels of contribution (infrastructure, code interfaces, analysis modules, documentation, community engagement) it has scaled to contributions from 50 different people, with an active development process and user community from across the globe.

Originally applied to astrophysical simulations, yt has been extended to work with radio telescope data, seismological simulations, high energy-density physics data, and nuclear engineering simulations.

[PyCUDA](#)



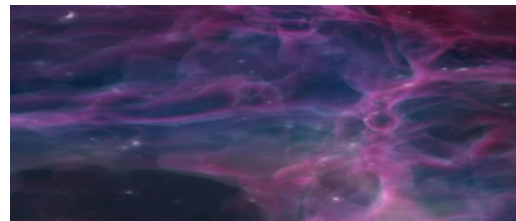
PyCUDA[2] is an open source python package developed by Andreas Klöckner. This package offers a rich variety of tools that both simplify the use of CUDA and make it far more synergistic with Python. Since our research primarily entails using a python interpreter in order to facilitate computation, this method saves us from needing to write interoperability code between CUDA and Python. Preventing this step gives us more time to focus on our ray-tracing algorithms. In addition, strictly python computation would have not allowed us to take advantage of the GPU, and would therefore hinder our results.

Specifically this method allows us to focus on the development of CUDA kernel functions only, allowing us to create a diverse visualization library for use with yt.

[Volume Rendering with CUDA](#)

In the UCSC Astrophysics 3D Visualization lab, we are developing a yt module that performs volume rendering on an Nvidia graphics card. We use CUDA to create ray-casting algorithms that move the volume rendering of the numerous data structures yt can parse onto the graphics card allowing users to interact with the data in a 3D space in real time.

We've successfully implemented a software toolchain that allows a user to process data with yt and then perform volume rendering on CUDA cores. The results of the volume rendering are then passed to OpenGL allowing users to interact with the data in real-time. Using a Quadro K6000, which was donated by Nvidia, 4.5 Gbytes of data was displayed at 50 frames per second. Alternatively, the results of the CUDA volume renderer can be displayed remotely in a users browser via the ipython notebook. This allows users to take advantage of GPU clusters to create high resolution videos.



Our results show that providing yt with a graphics suite that takes advantage of Nvidia CUDA cores will be a powerful contribution to the scientific community. We hope to extend visualization support to numerous simulation codes. Once completed, our module will equip yt users with stunning visuals of star birth, galaxy formation, the cosmic web of dark matter, and many other astrophysical phenomena. We believe both the public and the scientific community will benefit greatly from these advances in visualization software. Because yt is used for scientific work providing high-quality visuals, our work will enable easier public outreach.

IPython Notebook



A real time streaming app that is embedded directly into IPython [3] Notebook is possible with the latest websocket protocol. This allows the user to make renderings on a remote server and stream images of those renderings to a window inside of IPython Notebook in real time. By simply calling Stream() on the CUDA context an HTML5 canvas is dropped into the output window and live images rendered on the server are displayed to the canvas. The mouse position is passed back to the server to update the rendering allowing the user to drag their mouse and rotate the data in real time. We have this kind of real time interaction with OpenGL, however, it requires you to be working on the machine locally. Streaming directly to the IPython Notebook allows the user to work from anywhere with an internet connection.

Streaming is done by forwarding an extra port when establishing a connection to the server via ssh; the IPython Notebook server is then started normally. From inside a notebook, the user creates a CUDA rendering context and then calls Stream() on that object. Stream() creates a new thread that starts a websocket server for communication with the browser while the parent thread returns a small piece of HTML5 and Javascript to set up the canvas inside of the IPython Notebook output window. The browser then sends a request to the server with the current mouse position, relative to the canvas window, and the server makes a rendering, encodes it into a jpeg image, serializes it for websocket communication and sends it to the browser. Finally, the browser deserializes the incoming jpeg and writes the image to the HTML5 canvas. The entire streaming system is all done with HTML and Javascript on the client side and uses Tornado web framework in Python on the server side.

PyOpenGL

Because our volume renderer relies on yt and PyCuda we naturally use a python implementation of OpenGL to create an interactive user experience. PyOpenGL[4] is capable of binding a pointer supplied by PyCuda to a texture representing the results of the ray casting. Then the user can use i/o devices such as a keyboard or mouse to control the view of the data.

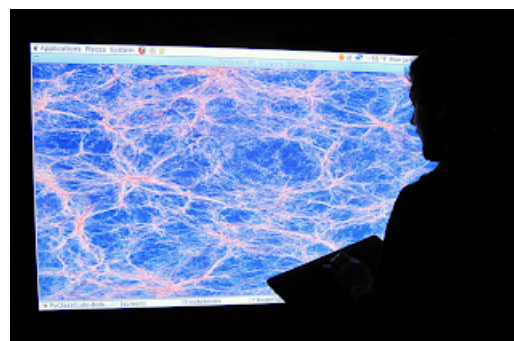
The PyCuda to PyOpenGL interop allows the data to persist in device texture memory. The result of the ray casting through the data can be displayed at 50 fps on a 1080p screen. This gives the user the feeling that they are exploring the data in real time. This also allows the user to highlight interesting features of the data by changing color and brightness.

Stereoscopic Rendering

With the latest high performance Nvidia cards, such as the Quadro K6000, interactive stereoscopic volume rendering can be achieved in real-time. Though the frame rate suffers dropping to 20 fps from 50 fps, the user can still manipulate the data. Stereoscopic rendering gives viewers another piece of information to observe in his or her data. Combined with rotations this can reveal new information about three dimensional structures not previously seen in a two dimensional slice.

References:

[1] Turk, M. J., Smith, B. D., Oishi, J. S., Skory, S., Skillman, S. W., Abel, T., & Norman, M. L. "yt: A multi-code analysis toolkit for astrophysical simulation data." The Astrophysical Journal



Supplement Series 192.1 (2011): 9.

[2] PyCuda Main Page, <http://mathematician.de/software/pycuda/>, retrieved January 27, 2014

[3] IPython Main Page, <http://ipython.org/notebook.html>, retrieved January 27, 2014

[4] PyOpenGL Main Page, <http://pyopengl.sourceforge.net/>, retrieved January 27, 2014

Thanks to:

Physics Professor Joel Primack, Astronomy & Astrophysics Professor Enrico Ramirez-Ruiz, University of California High-performance AstroComputing Center (UC-HiPACC), Matt Turk and the yt community, Nvidia for the donation of a Quadro K6000, and Nathan Goldbaum